UNIVERSITÉ ABDELMALEK ESSAÂDI

ENSAH - École Nationale des Sciences Appliquées Al Hoceima

RAPPORT DE PROJET
ARCHITECTURE WEB MVC

Module : Applications Web Avancées

Mise en place d'une application de gestion de produits

Réalisé par :

Ayoub NASSIH

Encadré par :

M. CHERRADI

Filière : Transformation digitale et IA

Année Universitaire : 2025-2026

0 1

1 Introduction Générale **2**

2 Cahier des Charges et Spécifications **2**
2.1 Spécifications Fonctionnelles 3
2.2 Spécifications Techniques 3

3 Conception et Modélisation **3**
3.1 Diagramme des Cas d'Utilisation 3

4 Architecture Détaillée (Le Motif MVC) **4**
4.1 Schéma Explicatif de l'Architecture 5

5 La Couche Modèle : Encapsulation et Persistance (DAO) **7**
5.1 Création de l'entité Produit (JavaBean) 7
5.2 Le contrat d'Interface (ProduitDAO) 7
5.3 L'Implémentation du DAO 8

6 La Couche Métier (Les Services) **9**

7 La Couche Contrôleur : Servlets et Authentification **10**
7.1 Gestion des Formulaires (Méthode POST) 10
7.2 Sécurisation, Sessions et Contrôle d'Accès 11

8 La Couche Vue : Rendu et Fichiers JSP **11**
8.1 Itération et Affichage Dynamique 12

9 Interfaces Graphiques et Résultat Fonctionnel **13**
9.1 Briques d'Authentification 13
9.2 Administration du Catalogue et Tableaux de bord 13

10 Conclusion et Perspectives d'Évolution **14**

1 1

Le développement d'applications web robustes et évolutives repose aujourd'hui fondamentalement sur des motifs de conception solides et éprouvés par l'industrie. Dans le cadre de notre formation d'ingénieur en **Transformation digitale et Intelligence Artificielle**, et plus spécifiquement au sein du module **"Applications web avancées avec Java et Spring"**, il nous a été confié le développement d'une application professionnelle de gestion.

L'objectif central de ce projet est de concevoir, modéliser et implémenter de A à Z une application web de gestion de produits (incluant l'authentification et le contrôle d'accès) en s'appuyant strictement et rigoureusement sur le pattern architectural **MVC (Modèle - Vue - Contrôleur)** en utilisant l'écosystème natif Java EE (Servlets, JSP, JavaBeans).

Ce rapport détaille la totalité du cycle de vie du développement : de l'analyse des besoins à l'implémentation algorithmique, en passant par la justification des choix architecturaux et la présentation du rendu final.

Objectifs & Missions

- Appréhender et maîtriser les fondements du web dynamique avec Java EE.
- Séparer rigoureusement la logique métier, l'accès aux données et l'affichage (Pattern MVC).
- Implémenter des patrons de conception fondamentaux tels que le **DAO** (Data Access Object) et le **Singleton**.
- Sécuriser l'application via un système de gestion de **Sessions HTTP** (Filtres/-Redirections).
- Créer des vues dynamiques avec JSP (JavaServer Pages) encapsulées pour prévenir l'accès direct.

2 1

Avant d'entamer la phase de codage, il est impératif de définir le périmètre fonctionnel et technique de l'application.

2.1 1Spécifications Fonctionnelles

L'application doit répondre aux besoins utilisateurs suivants :

— **Gestion de l'authentification :** Un utilisateur doit pouvoir s'inscrire, se connecter et se déconnecter. Le système gère les sessions actives.

— **Sécurisation des ressources :** Aucune action de visualisation ou de modification des données "Produit" ne doit être permise sans une session valide.

— **Opérations CRUD sur les Produits :** Une fois connecté, l'utilisateur a accès à un tableau de bord lui permettant de :

→ **C**réer un nouveau produit (Nom, Description, Prix).

→ **R**écupérer et lister l'ensemble des produits disponibles.

→ **U**pdater (Mettre à jour) les informations d'un produit existant.

→ **D**elete (Supprimer) un produit du catalogue.

2.2 1Spécifications Techniques

— **Langage Core :** Java (JDK 8 ou supérieur).

— **Serveur d'application :** Apache Tomcat (Conteneur de Servlets/JSP).

— **Architecture :** MVC 2 pur (Servlets en tant que contrôleurs, JSP en tant que vues).

— **Modèle de persistance :** Gestion en mémoire (Collections Collections/Lists Java) via le pattern DAO, pour permettre une scalabilité future vers une base de données (MySQL/PostgreSQL) avec JDBC sans modifier la logique métier.

3 1

La phase de modélisation est assurée par le langage UML (Unified Modeling Language) afin de cartographier de manière standardisée les interactions de l'utilisateur avec notre système.

3.1 1Diagramme des Cas d'Utilisation

Le diagramme ci-dessous illustre le périmètre d'action du visiteur (non connecté) comparé à celui de l'utilisateur (authentifié). L'inclusion (*«include»*) démontre que toutes les actions sur les produits exigent impérativement l'authentification.

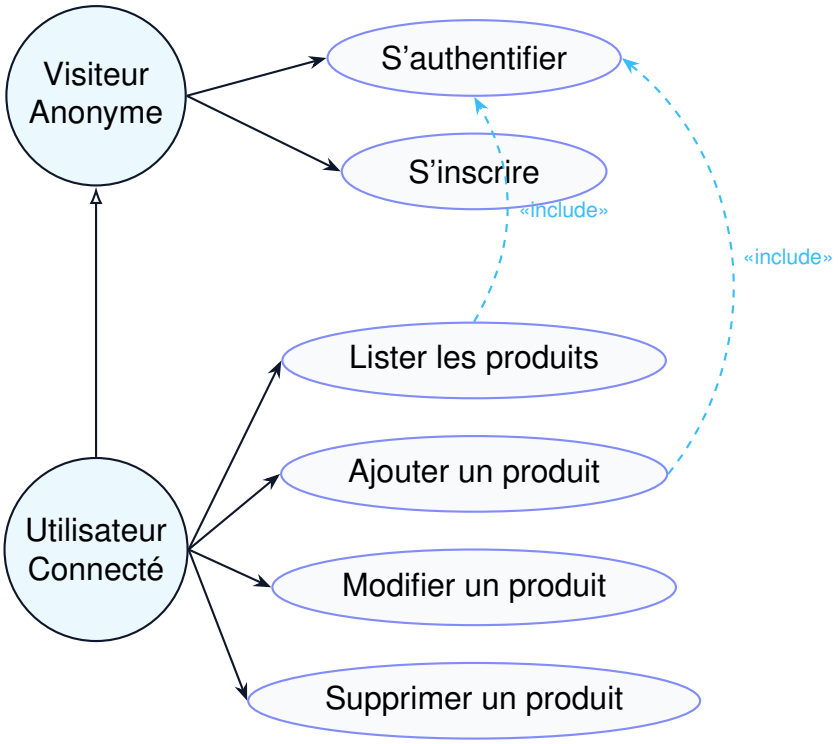

FIGURE 1 – Diagramme des cas d'utilisation du système de gestion.

4 1

Un projet de cette envergure nécessite une séparation drastique des responsabilités pour garantir sa **maintenabilité** et sa **testabilité**. Le choix de l'architecture Modèle-Vue-Contrôleur répond parfaitement à cette problématique.

4.1 1Schéma Explicatif de l'Architecture

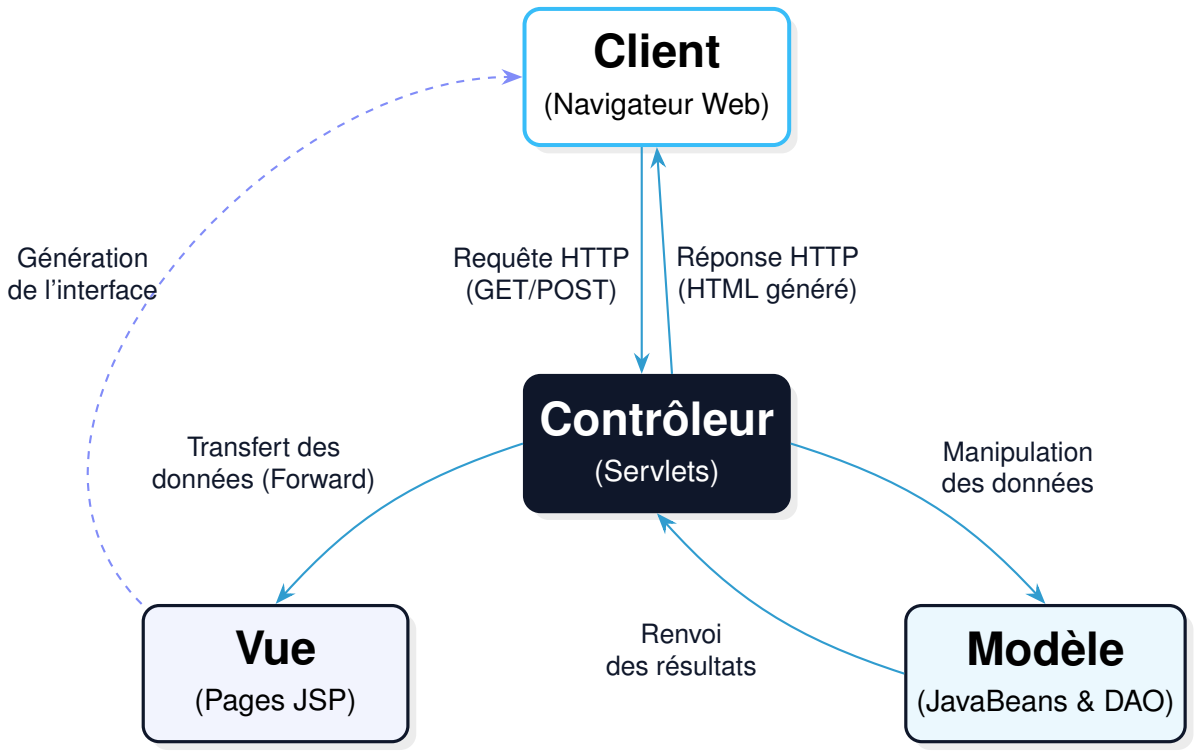

FIGURE 2 – Diagramme explicatif premium de l'architecture MVC.

L'arborescence matérielle de notre IDE découle logiquement de cette structuration. L'organisation des packages (`dao`, `services`, `web`, `auth`) est le reflet direct de notre architecture.

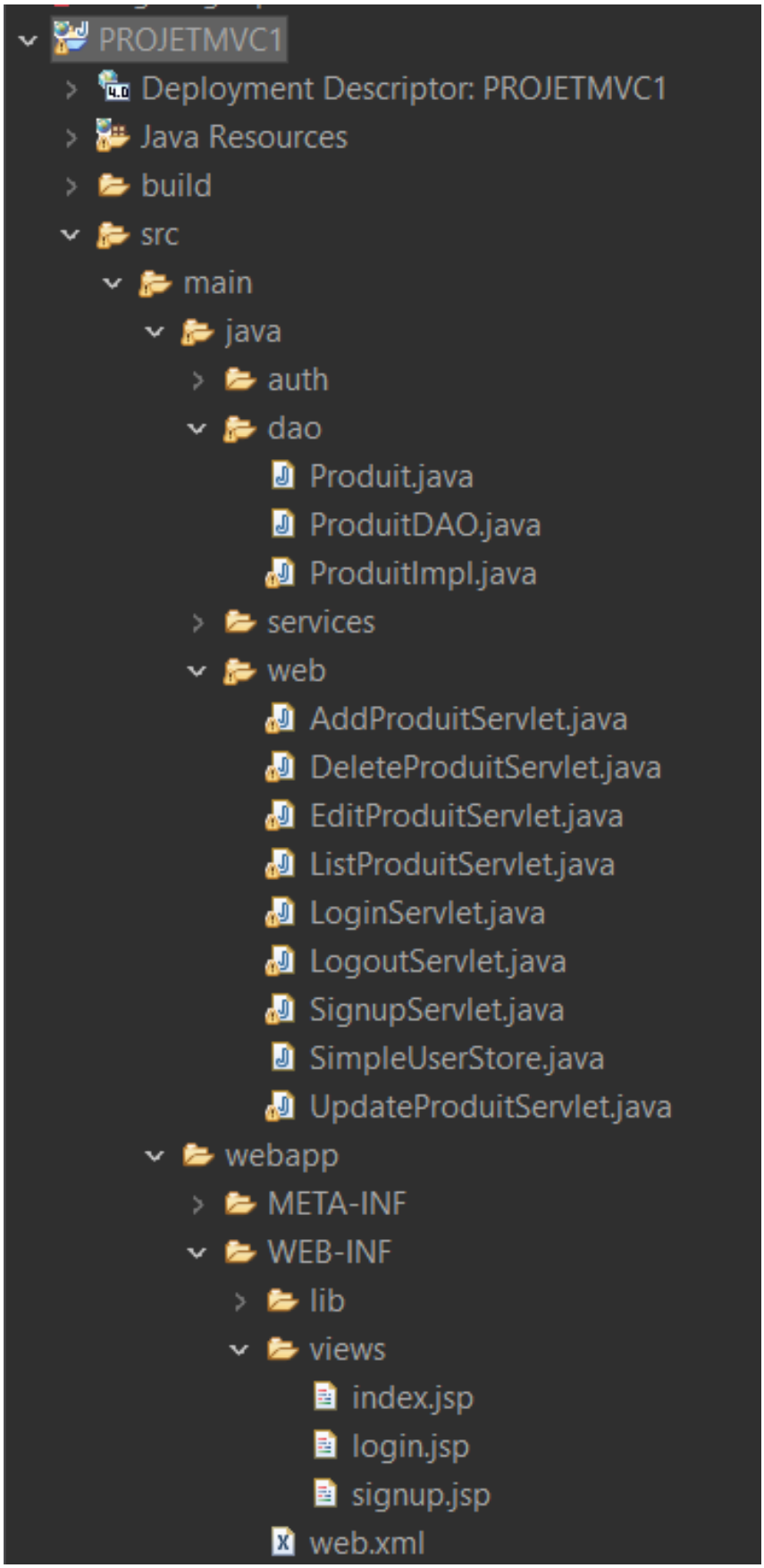

FIGURE 3 – Arborescence globale du projet sous l'IDE Eclipse.

5 1

La couche modèle est le cœur fonctionnel de l'application. Elle représente le "M" du MVC et est sciemment divisée en deux sous-parties : les Entités (JavaBeans) et la couche d'accès aux données (Data Access Object).

5.1 1Création de l'entité Produit (JavaBean)

Un JavaBean est une classe simple qui encapsule plusieurs objets en un seul ensemble. Notre entité `Produit.java` respecte scrupuleusement les conventions Java (attributs privés, constructeurs, Getters et Setters).

Entité Modèle | java

```java
package dao;

public class Produit {
    private Long idProduit;
    private String nom;
    private String description;
    private Double prix;

    public Produit(String nom, String description, Double prix) {
        this.nom = nom;
        this.description = description;
        this.prix = prix;
    }

    public Long getIDproduit() { return idProduit; }
    public void setIDproduit(Long idProduit) { this.idProduit = idProduit; }
    public String getNom() { return nom; }
    public void setNom(String nom) { this.nom = nom; }
}
```

5.2 1Le contrat d'Interface (ProduitDAO)

Le design pattern **DAO** impose la création d'une interface définissant les méthodes d'interaction avec le stockage. Si un jour le stockage passe d'une liste en mémoire à

une base de données MySQL, le Contrôleur ne s'en rendra même pas compte.

Interface DAO | java

```java
package dao;
import java.util.List;

public interface ProduitDAO {
    public void addProduit(Produit p);
    public void deleteProduit(Long Id);
    public void updateProduit(Produit p);
    public Produit getProduitById(Long Id);
    public List<Produit> getAllProducts();
}
```

5.3 1L'Implémentation du DAO

L'implémentation `ProduitImpl.java` gère actuellement les données dans la mémoire vive du serveur via une instance dynamique de `ArrayList`.

Implémentation de gestion mémoire | java

```java
package dao;
import java.util.ArrayList;
import java.util.List;

public class ProduitImpl implements ProduitDAO {
    private List<Produit> Produits = new ArrayList<Produit>();
    private long currentId = 0;

    public void init() {
        System.out.println("Systeme initialise avec donnees par defaut");
        addProduit(new Produit("Camera 1", "Canon R5", 23000.0));
        addProduit(new Produit("Camera 2", "Sony A6400", 9000.0));
    }

    public void addProduit(Produit p) {
        currentId++;
        p.setIDproduit(currentId);
        Produits.add(p);
    }
```

```
}
```

Concept / Remarque

Le pattern **DAO** centralise et encadre toutes les opérations de type CRUD. La classe `ProduitImpl` implémente le contrat fixé par `ProduitDAO`, assurant une abstraction totale de la source de données pour la couche logicielle supérieure.

6 1

Insérée entre le DAO et les Contrôleurs, la couche métier gère la logique applicative. Dans notre architecture, `ProduitMetierImpl.java` utilise un Singleton pour s'assurer qu'une seule instance interagit avec notre base de données "en mémoire" durant toute la durée de vie du serveur Tomcat.

Singleton dans le service métier | java

```java
package services;
import dao.ProduitDAO;
import dao.ProduitImpl;

public class ProduitMetierImpl implements ProduitMetier {
    private static ProduitMetierImpl instance;
    private ProduitDAO dao;

    private ProduitMetierImpl() {
        dao = new ProduitImpl();
        ((ProduitImpl)dao).init();
    }

    public static synchronized ProduitMetierImpl getinstance() {
        if (instance == null) {
            instance = new ProduitMetierImpl();
        }
        return instance;
    }
}
```

7 1

Les **Servlets** jouent le rôle critique de Chef d'Orchestre. Conformément au protocole HTTP, ils interceptent les requêtes via `doGet` ou `doPost`, interrogent la couche métier, puis "forward" ou "redirect" la réponse vers la vue appropriée.

7.1 1Gestion des Formulaires (Méthode POST)

La classe `AddProduitServlet.java` permet l'ajout de nouvelles entités. Elle extrait les paramètres HTTP injectés par le formulaire JSP, gère les conversions de types (String vers Double), et fait appel au métier.

Extrait : Ajout d'un Produit en Servlet | java

```java
package web;

@WebServlet("/add")
public class AddProduitServlet extends HttpServlet {
    private static final ProduitMetier metier =
        ProduitMetierImpl.getinstance();

    @Override
    protected void doPost(HttpServletRequest req,
        HttpServletResponse resp)
            throws ServletException, IOException {

        String nom = req.getParameter("nom");
        String description = req.getParameter("description");
        String prixStr = req.getParameter("prix");

        try {
            Double prix = Double.parseDouble(prixStr);
            Produit nouveauProduit = new Produit(nom,
                description, prix);
            metier.addProduit(nouveauProduit);

            resp.sendRedirect(req.getContextPath() + "/list");
        } catch (NumberFormatException e) {
            req.getRequestDispatcher("/WEB-INF/views/addProduit.
                jsp").forward(req, resp);
        }
```

```java
    }
}
```

7.2 1Sécurisation, Sessions et Contrôle d'Accès

L'application intègre un espace sécurisé. Lors du passage dans `LoginServlet.java`, si les credentials sont corrects (vérifiés par notre `SimpleUserStore`), nous générons une Session serveur et y attribuons notre token.

Création de session et vérification | java

```java
HttpSession session = req.getSession();
session.setAttribute("connectedUser", nomUtilisateur);
resp.sendRedirect(req.getContextPath() + "/list");

@Override
protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

    if (req.getSession().getAttribute("connectedUser") == null) {
        resp.sendRedirect(req.getContextPath() + "/login");
        return;
    }

    List<Produit> liste = metier.getAllProduits();
    req.setAttribute("listeProduits", liste);
    req.getRequestDispatcher("/WEB-INF/views/index.jsp").forward(req, resp);
}
```

8 1

Les vues de l'application ont été développées à l'aide de JSP (JavaServer Pages) tout en encapsulant le code de manière optimale dans le dossier `/WEB-INF/views/` pour empêcher tout accès direct non autorisé aux fichiers de présentation (via une

saisie d'URL directe par un utilisateur malveillant).

8.1 1Itération et Affichage Dynamique

Le rendu du tableau contenant la liste des produits montre comment le code Java (via scriptlets ou tags JSTL) est injecté pour dynamiser les interfaces front-end HTML.

Extrait du fichier index.jsp (Vue Principale) | html

```html
<table class="table table-bordered">
    <thead>
        <tr>
            <th>ID</th>
            <th>Nom de l'article</th>
            <th>Description</th>
            <th>Prix (MAD)</th>
            <th>Actions</th>
        </tr>
    </thead>
        <%
            List<Produit> produits = (List<Produit>) request.
                getAttribute("listeProduits");
            if(produits != null && !produits.isEmpty()) {
                for(Produit p : produits) {
        %>
        <tr>
            <td><%= p.getIDproduit() %></td>
            <td><%= p.getNom() %></td>
            <td><%= p.getDescription() %></td>
            <td><%= p.getPrix() %> MAD</td>
            <td>
                <a href="edit?id=<%= p.getIDproduit() %>" class=
                    "btn btn-warning">Editer</a>
                <a href="delete?id=<%= p.getIDproduit() %>"
                    class="btn btn-danger">Supprimer</a>
            </td>
        </tr>
        <%
                }
            } else {
        %>
        <tr><td colspan="5">Aucun produit disponible.</td></tr>
        <% } %>
    </tbody>
</table>
```

9 1

Après l'intégration d'un framework CSS (bootstrap ou custom design) sur nos JSP, l'application finale propose des interfaces épurées, "User Friendly" et ergonomiques.

9.1 1Briques d'Authentification

Le processus démarre impérativement par la page de connexion. En l'absence de compte, le visiteur est fluidement orienté vers la page de création de profil.

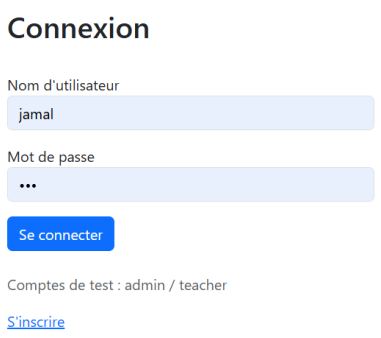

FIGURE 4 – Interface de Connexion : `login.jsp`

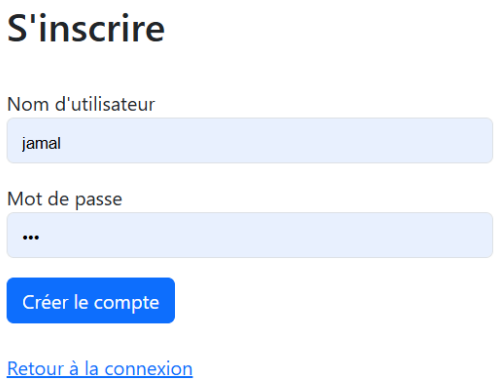

FIGURE 5 – Interface d'Inscription : `signup.jsp`

9.2 1Administration du Catalogue et Tableaux de bord

Une fois le filtre d'authentification passé, l'utilisateur est gratifié de l'accès au tableau central. Depuis cette vue, toutes les routes menant aux Servlets ajout, modification et suppression sont accessibles.

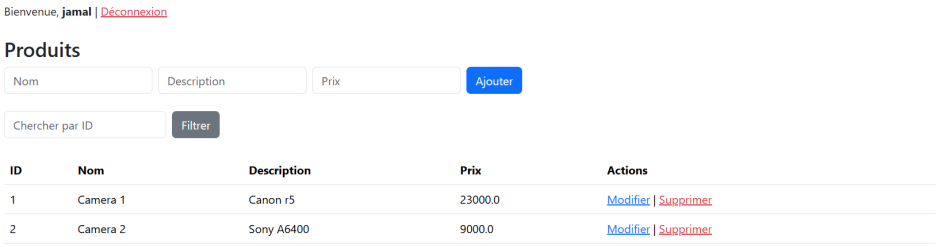

FIGURE 6 – Interface Principale d'Administration : Liste des Produits.

FIGURE 7 – Formulaires dynamiques injectant les paramètres HTTP en POST.

10 1

Le présent travail a mis en évidence l'extrême robustesse et flexibilité de l'architecture **MVC (Modèle-Vue-Contrôleur)** appliquée à l'écosystème Java EE (Servlets/-JSP). Ce projet n'a pas seulement été un exercice de programmation, mais une véritable démarche d'ingénierie logicielle couvrant la séparation des responsabilités informatiques.

La couche modèle via les JavaBeans et les interfaces **DAO** ont rendu notre code totalement indépendant vis-à-vis du mode de persistance des données. L'exploitation intensive des **Servlets** comme uniques points d'entrée (Contrôleurs) a assuré un contrôle draconien de la sécurité liée aux sessions avant la présentation des interfaces JSP (`WEB-INF` lock).

1Perspectives Futures

Si nous devions transposer ce projet éducatif à l'échelle d'une application industrielle pour une grande entreprise, plusieurs perspectives d'amélioration drastiques seraient intégrées :

1. **Migration vers des Frameworks Haut Niveau :** Le remplacement de notre configuration "Servlets Manuels" par le framework **Spring Boot**, ce qui réduirait massivement le "code boilerplate" grâce à l'inversion de contrôle (IoC) et l'injection de dépendances (DI).
2. **ORM pour la Persistance :** Intégration de **Hibernate** et **Spring Data JPA** afin de mapper automatiquement nos JavaBeans (Entités) à des tables relationnelles (PostgreSQL ou MySQL) de manière transparente.
3. **Détachement Client-Serveur (API REST) :** Passer d'une architecture de rendu serveur (JSP) à une architecture désolidarisée où Java agirait uniquement comme une **API RESTful** (rendant du JSON), et où une technologie front-end moderne (Angular, React, VueJS) se chargerait de composer graphiquement la page côté navigateur client.

Cette conception démontre avec clarté que les acquis de ce cours en web avancé sont le socle indispensable à la compréhension et à la maîtrise de tout l'ingénierie web moderne.